

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ЖЕЛЕЗНОДОРОЖНОГО ТРАНСПОРТА
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Петербургский государственный университет путей сообщения
Императора Александра I»
(ФГБОУ ВО ПГУПС)**

Факультет «Автоматизация и интеллектуальные технологии»
Кафедра «Информатика и информационная безопасность»

Лабораторная работа № 3.1
по дисциплине
«Разработка защищенных приложений под ОС семейства Unix»
«Совместное применение системных вызовов fork() и exec()»

Выполнил обучающийся

Курс 4

Группа КИБ-312

Б.Г. Народицкий

Проверил

А. И. Загайнов

Санкт-Петербург
2026

Оценка лабораторной работы

Показатель оценивания	Критерии оценивания	Шкала оценивания
Качество выполнения задания, отчета и защиты по лабораторной работе	Нет замечаний	4
	Зафиксирован один из следующих недочетов: – допущены ошибки при выполнении задания, но в целом задание выполнено; – даны неверные ответы на часть контрольных вопросов; – имеются отдельные недостатки в оформлении отчета; – работа сдана с опозданием на две недели и более.	3
	Зафиксировано два недочета из перечисленных в предыдущем пункте	2
	Допущено значительное количество неточностей или задание не выполнено, не даны ответы на контрольные вопросы	0
Количество баллов по результатам проверки и защиты лабораторной работы (вписывается преподавателем)		

1. Задание

Разработайте программу, которая создает дочерний процесс и запускает в рамках созданного дочернего процесса одну из стандартных утилит ОС Linux. При наличии ошибок, связанных с созданием дочернего процесса или запуском утилиты, программа должна выводить соответствующие сообщения.

2. Выполнение

В ходе выполнения лабораторной работы разработано программное обеспечение, где запускается процесс в котором запускается ещё один дочерний процесс

Исходный текст разработанного программного обеспечения выглядит следующим образом:

```
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    // По умолчанию запускаем pwd, но можно передать свою команду в аргументах
    const char *cmd = (argc > 1) ? argv[1] : "pwd";

    printf("Родитель (PID %d): готовлюсь запустить утилиту '%s'\n", getpid(), cmd);

    pid_t pid = fork();

    // 1. Обработка ошибки создания дочернего процесса (требование методички)
    if (pid < 0)
    {
        perror("Ошибка fork()");
        exit(1);
    }

    if (pid == 0)
    {
        // Мы в дочернем процессе
        printf("Потомок (PID %d): запускаю '%s'\n", getpid(), cmd);

        // exeslp ищет утилиту в PATH.
        // Первый аргумент - имя файла, второй - argv[0] (обычно совпадает), далее
        NULL
        exeslp(cmd, cmd, NULL);

        // ВАЖНО: Если exeslp успешен, код ниже НИКОГДА не выполнится.
        // Если мы здесь, значит произошла ошибка (например, утилита не найдена).
        perror("Ошибка exeslp()");
        exit(127); // Стандартный код ошибки "команда не найдена"
    }

    // Мы в родительском процессе
    int status;
```

```

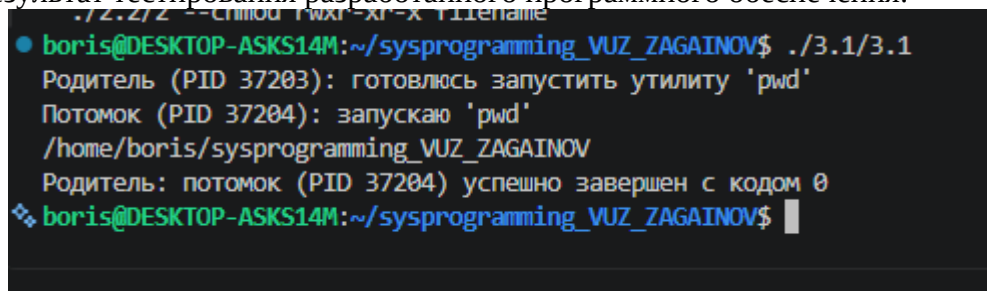
// 2. Ждем завершения КОНКРЕТНОГО дочернего процесса
if (waitpid(pid, &status, 0) < 0)
{
    perror("Ошибка waitpid()");
    return 1;
}

// 3. Анализ того, как завершился потомок
if (WIFEXITED(status))
{
    printf("Родитель: потомок (PID %d) успешно завершен с кодом %d\n",
        pid, WEXITSTATUS(status));
}
else
{
    printf("Родитель: потомок (PID %d) завершен аварийно\n", pid);
}

return 0;
}

```

Результат тестирования разработанного программного обеспечения:



```

./3.1/3.1
Родитель (PID 37203): готовлюсь запустить утилиту 'pwd'
Потомок (PID 37204): запускаю 'pwd'
/home/boris/sysprogramming_VUZ_ZAGAINOV
Родитель: потомок (PID 37204) успешно завершен с кодом 0
❖ boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV$

```